

GetChallenges Class

Apex Code

```
public with sharing class GetChallenges {
    public class Input {
        @InvocableVariable(description='The Challenge Status of the Challenge to search for')
        public String status;
        @InvocableVariable(description='The Start Date of the Challenge to search for')
        public Date startDate;
        @InvocableVariable(description='The End Date of the Challenge to search for')
        public Date endDate;
        @InvocableVariable(description='The Challenge ID of the Challenge to search for')
        public String id;
        @InvocableVariable(description='The Name of the Challenge to search for')
        public String name;
        @InvocableVariable(description='The Manager of the Challenge to search for')
        public String manager;
        @InvocableVariable(description='The Role of the Participant of the Challenge to search for')
        public String participantRole;
        @InvocableVariable(description='The Name of the Participant of the Challenge to search for')
        public String participantName;
    }

    public class Output {
        @InvocableVariable(description='List of Challenge Objects found')
        public List<Challenge__c> challenges;
        @InvocableVariable(description='Result message of the search')
        public String message;

        public Output(List<Challenge__c> challenges, String message) {
            this.challenges = challenges;
            this.message = message;
        }
    }

    @InvocableMethod(label='Get Challenges' description='Query challenges based on various criteria')
    public static List<Output> getChallenges(List<Input> inputList) {
        List<Output> results = new List<Output>();
        if (inputList == null || inputList.isEmpty()) {
            results.add(new Output(new List<Challenge__c>(), 'No input provided'));
            return results;
        }

        Input queryInput = inputList[0];
```

```

List<Challenge__c> challenges = ChallengeHelper.queryChallenges(
    queryInput.status,
    queryInput.startDate,
    queryInput.endDate,
    queryInput.id,
    queryInput.name,
    queryInput.manager
);

// Delegate participant filtering to ChallengeHelper
List<Challenge__c> filteredChallenges =
ChallengeHelper.filterChallengesByParticipant(
    challenges,
    queryInput.participantRole,
    queryInput.participantName
);

if (filteredChallenges.isEmpty()) {
    results.add(new Output(new List<Challenge__c>(), 'No challenges found'));
} else {
    results.add(new Output(filteredChallenges, 'Challenges retrieved
successfully'));
}

return results;
}
}

```

Methods

getChallenges(inputList)

INVOCABLEMETHOD

Signature

```
public static List<Output> getChallenges(List<Input> inputList)
```

Parameters

Name	Type	Description
inputList	List<Input>	

Return Type

List<Output>

Classes

Input Class

Fields

status

INVOCABLEVARIABLE

SIGNATURE

public status

TYPE

String

startDate

INVOCABLEVARIABLE

SIGNATURE

public startDate

TYPE

Date

endDate

INVOCABLEVARIABLE

SIGNATURE

public endDate

TYPE

Date

id

INVOCABLEVARIABLE

SIGNATURE

public id

TYPE

String

name

INVOCABLEVARIABLE

SIGNATURE

public name

TYPE

String

manager

INVOCABLEVARIABLE

SIGNATURE

public manager

TYPE

String

participantRole

INVOCABLEVARIABLE

SIGNATURE

public participantRole

TYPE

String

participantName

INVOCABLEVARIABLE

SIGNATURE

public participantName

TYPE

String

Output Class

Fields

challenges

INVOCABLEVARIABLE

SIGNATURE

public challenges

TYPE

List<Challenge__c>

message

INVOCABLEVARIABLE

SIGNATURE

```
public message
```

TYPE

String

Constructors

```
Output(challenges, message)
```

SIGNATURE

```
public Output(List<Challenge__c> challenges, String message)
```

PARAMETERS

Name	Type	Description
challenges	List<Challenge__c>	
message	String	